

Republic of Yemen

الجمهورية اليمنية

Ministry of Higher Education

وزارة التعليم العالي

Taiz University

جامعة تعز

Faculty of engineering & IT



Judicial Smart Justice System (JSJS)

Project report submitted as part of the requirements for obtaining a Bachelor of Technology degree in Software Engineering

Engineers:

BASSAM DALIL MOHAMMED RADMAN
ABDULWAHID NABIL MOHAMMED ALI AL-EMAD
MUSHEER QAID ALI HASAN GAMIL

Supervised By:

Eng. Anas Al-sabri

Yemen – Taiz

2025 – 2026

نموذجًا أوليًا لنظام عدالة ذكي يهدف إلى رقمنة إجراءات تسجيل القضايا ومتابعتها داخل بيئة قضائية. يركز النظام على قضايا JSJS يقدم مشروع يوفر المشروع، **Laravel** الأحوال الشخصية والقضايا المدنية المرتبطة الأطراف، الوقائع، الأسانيد القانونية، المرفقات، وحالة القضية. ومن خلال بنية صفحات إدخال، قوائم عرض، آليات بحث، وإمكانية تعديل وحذف الملفات القضائية بطريقة منظمة

الفكرة الأساسية للمشروع هي تحويل الإجراءات اليدوية المتفرقة إلى منصة مركزية تساعد موظفي المحكمة والإدارة والقضاة على الوصول إلى البيانات، بسرعة، وتقليل ضياع المعلومات، وتحسين قدرة الإدارة على قياس حجم القضايا المتراكمة. ويظهر ذلك من خلال وجود نماذج لإضافة قضية جديدة إضافة إلى لوحة خاصة بالقضاة تعرض مؤشرات **PDF** حفظ بيانات المدعي والمدعى عليه، تفاصيل الزواج عند الحاجة، الشهود، المرفقات بصيغة

القيمة الرئيسية

القيمة الأبرز للمشروع ليست في تخزين البيانات فقط، بل في ربط بيانات القضية بالأطراف والمرفقات والمؤشرات التحليلية؛ وهذا يجعل النظام أقرب إلى أداة متابعة قضائية لا مجرد سجل إلكتروني

محتوى مختصر	محتوى
تطبيق ويب لإدارة القضايا القضائية	نوع النظام
Blade وواجهات PHP مع Laravel	الإطار المستخدم
Migrations حسب إعدادات البيئة، مع جداول منظمة عبر SQL Server	قاعدة البيانات
تسجيل القضايا، البحث، التعديل، الإحصاءات، وتحليلات مساعدة	المخرجات

مشكلة المشروع وأهدافه

تعاني البيانات القضائية التقليدية من بطء في الوصول إلى ملفات القضايا، وتكرار إدخال البيانات، وصعوبة تتبع حالة الدعوى، إضافة إلى صعوبة كمقترح رقمي يخفف الاعتماد JSJS استخراج مؤشرات إدارية مثل عدد القضايا الجديدة أو القضايا قيد النظر أو القضايا المحكومة. لذلك يأتي مشروع على الورق ويدعم اتخاذ القرار الإداري والقضائي

يعالج المشروع مشكلة تعدد مصادر البيانات القضائية من خلال نموذج موحد لإضافة القضية. فالملف القضائي في النظام لا يتكون من رقم قضية فقط، بل يشمل نوع القضية، الوقائع، الأسانيد القانونية، بيانات الأطراف، التفاصيل الخاصة مثل بيانات الزواج والشهود، وحفظ ملخص أو ملف مرفق. هذا

التجميع يساعد على بناء سجل واضح ومتناهي للقضية

الهدف	الأثر المتوقع
رقمنة تسجيل القضايا	تقليل الاعتماد على النماذج الورقية وتسهيل استرجاع البيانات
تسريع البحث والمتابعة	البحث برقم القضية أو اسم أحد الأطراف بدل البحث اليدوي
توحيد بيانات الأطراف	ربط المدعي والمدعى عليه بالقضية بطريقة منظمة
دعم القاضي بمؤشرات	عرض إحصاءات تساعد على فهم الضغط وتراكم القضايا
رفع جودة الإدارة	تسهيل إدارة المستخدمين ومتابعة صلاحيات الدخول

من أهداف المشروع كذلك إبراز دور الذكاء الاصطناعي بشكل أولي من خلال تقديم توصيات وتحذيرات مبنية على نسب القضايا المتراكمة وأنواع ومحاكى أكثر من كونه نموذجًا ذكائيًا متقدمًا، إلا أنه يمثل خطوة جيدة لتوضيح rule-based القضايا الأكثر انتشارًا. ورغم أن التحليل الحالي يبدو الفكرة في مشروع تخرج أو عرض جامعي

نطاق النظام والمستخدمون المستهدفون

دورة حياة أولية ملف القضية منذ تسجيلها وحتى ظهورها في قائمة القضايا ولوحة القضية. ويشمل ذلك إدخال البيانات الأساسية، حفظ JSJS يغطي نطاق العلاقات بين القضية والأطراف، رفع المرفقات، وتغيير حالة القضية. كما يحتوي المشروع على أكثر من مدخل للنظام: مدخل للموظف أو المستخدم العادي، مدخل خاص بالقضية، ومدخل لوحة الادارة

المستخدم	الدور داخل النظام	أمثلة على العمليات
الموظف/المستخدم	إدخال القضايا والبحث عنها ومتابعتها	إضافة قضية، تعديل بيانات، حذف سجل، بحث برقم القضية.
القاضي	متابعة المؤشرات والتحليل والاطلاع على سجل القضايا	عرض إجمالي القضايا، القضايا قيد النظر، القضايا المحكومة
مدير النظام	إدارة المستخدمين ومراقبة الحسابات	إنشاء مستخدم، تعديل بيانات، حذف مستخدم
النظام نفسه	تنظيم البيانات وربط الجداول	إنشاء رقم قضية، حفظ مرفقات، ربط الأطراف بالقضية

حدود النطاق الحالي

النظام الحالي يركز على إدارة داخلية للقضايا ولا يظهر أنه يتضمن تكاملاً مباشراً مع منصات حكومية خارجية أو خدمات دفع أو توقيع إلكتروني. لذلك يمكن اعتباره نموذجاً عملياً قابلاً للتطوير

في أرقام القضايا، وتظهر أسماء محاكم TAIZ من خلال ملفات المشروع، يظهر أن التطبيق مصمم لقضايا مرتبطة بمحافظة تعز، حيث تُستخدم بادئة ومناطق يمنية في البيانات التجريبية. وهذا يعطي المشروع هوية وسياقاً محلياً واضحاً

المتطلبات الوظيفية وغير الوظيفية

المتطلبات الوظيفية هي العمليات التي يجب أن ينفذها النظام للمستخدم، أما المتطلبات غير الوظيفية فتتعلق بجودة النظام مثل الأمان، الأداء، سهولة الاستخدام، وقابلية الصيانة. ومن خلال الكود، يمكن استخراج مجموعة من المتطلبات المنفذة أو المقترحة

الدليل من المشروع		المتطلبات الوظيفية المنفذة
login/logout ومسارات AuthController		تسجيل الدخول للمستخدمين
cases/create ونموذج CaseController@store		إضافة قضية جديدة
CaseController@index. البحث برقم القضية أو اسم الطرف داخل		البحث في القضايا
مع دعم تحديث الأطراف والمرفقات CaseController@update		تعديل بيانات القضية
CaseController@destroy مع رد JSON أو إعادة توجيهه		حذف قضية
UserController وAdminController لعمليات CRUD.		إدارة المستخدمين
مع إحصاءات وتحليلات JudgeController@dashboard		لوحة تحليل للقضاة
توصية	التقييم الحالي	متطلب غير وظيفي
استبدالها بصلاحيات وأدوار داخل قاعدة البيانات	يوجد تسجيل دخول عادي، لكن بعض مداخل الإدارة تعتمد على كلمة مرور ثابتة	الأمان
عند توسع المشروع Policies و Services إضافة	يساعد على تنظيم الكود Laravel MVC استخدام	قابلية الصيانة
وفهارس بحث عند زيادة Pagination استخدام	الاستعلامات بسيطة ومناسبة للنموذج الحالي	الأداء
تحسين التحقق من المدخلات ورسائل الأخطاء	الواجهات عربية ومنظمة على مراحل	سهولة الاستخدام

وعن نماذج البيانات Controllers عن منطق التحكم Blade حيث يتم فصل الواجهات ،Laravel الشائع في MVC يعتمد المشروع على نمط .هذا الفصل يجعل المشروع أسهل في القراءة والتعديل، كما يسمح بتطوير كل طبقة دون التأثير الكبير على الطبقات الأخرى. Models.

الدور	المكونات	الطبقة
عرض النماذج، القوائم، لوحات التحكم، والتفاعل مع المستخدم.	Blade Views + CSS + JavaScript	واجهة المستخدم
استقبال الطلبات، التحقق، تنفيذ العمليات، وإرجاع النتائج.	AuthController, CaseController, AdminController, JudgeController	طبقة التحكم
تعريف الجداول والعلاقات بين القضايا والأطراف والشهود.	Models + Migrations	طبقة البيانات
تحديد الاتصال بقاعدة البيانات، الجلسات، الكاش البريد، وغيرها	.env + config	الإعدادات
إدارة ملفات الواجهة وتجهيزها للبناء	Vite + Tailwind + assets	الأصول والبناء

Web Browser



Laravel Routes (web.php)



Controllers: Case / Auth / Admin / Judge



Models: LegalCase / Party / User / Judge



Database Tables: cases, parties, users, judges, witnesses

PHP الذي يشغل خادم launcher.cs محمول، وملف PHP الذي يتضمن bin كما يحتوي المشروع على ملفات تشغيل محلية لبيئة ويندوز، منها مجلد المحلي على 127.0.0.1:8888 ويفتح النظام من المتصفح. هذه الإضافة مفيدة عند عرض المشروع على جهاز لا يحتوي على بيئة تطوير كاملة

تصميم قاعدة البيانات والعلاقات

parties وترتبط به جداول متعددة مثل cases بشكل منظم. الجدول المركزي هو migrations يظهر تصميم قاعدة البيانات في ملفات هذا التصميم مناسب لأن القضية الواحدة قد تحتوي على marriage_details, children, financial_claims, attachments, witnesses.

عدة أطراف أو شهود أو مرفقات، وقد تحتوي على تفاصيل إضافية حسب نوع القضية

أهم الحقول	الغرض	الجدول
username, name, email, password	حسابات مستخدمي النظام	users
case_number, case_type, facts, legal_reasons, status, judge_name	الملف الأساسي للقضية	cases
case_id, type, full_name, address, phone	أطراف القضية	parties
contract_date, guardian, place, dowry	تفاصيل الزواج عند القضايا الأسرية	marriage_details
case_id, name, identity_phone	بيانات الشهود	witnesses
full_name, national_id, status	سجل قضائي للمطابقة	judiciary_records
name, phone, specialization, court, status	بيانات القضاة	judges

العلاقات الأساسية

القضية لها عدة أطراف وعدة شهود، ولها تفاصيل زواج واحدة عند الحاجة، ويمكن أن ترتبط بمطالب مالية أو مرفقات. كما أن حذف القضية يؤدي إلى حذف onDelete(cascade). البيانات التابعة بفضل

يعطي مرونة في حفظ أكثر من سند قانوني للقضية الواحدة. وهذا مناسب لأن الدعوى قد تستند إلى أكثر من مادة JSON كحقل legal_reasons تخزين قانونية، مثل الفسخ، النفقة، الحضانة، أو غيرها

بمسار واضح يبدأ من الدخول إلى النظام، ثم الانتقال إلى لوحة التحكم، ثم البحث عن قضية أو إنشاء قضية جديدة. نموذج إنشاء JSJS يمر المستخدم داخل القضية مقسم إلى مراحل، وهذا يسهل على المستخدم إدخال البيانات تدريجيًا بدل وضع كل الحقول في صفحة طويلة غير منظمة

- 1) دخول المستخدم إلى النظام
- 2) الانتقال إلى لوحة التحكم
- 3) اختيار: إضافة قضية أو البحث عن قضية
- 4) إدخال بيانات المحكمة ونوع القضية
- 5) إدخال بيانات المدعي والمدعى عليه
- 6) إضافة تفاصيل الزواج والشهود عند الحاجة
- 7) كتابة الوقائع والأسانيد القانونية
- 8) أو كتابة ملخص مرفق PDF رفع ملف
- 9) حفظ القضية وإرجاع رسالة نجاح

المخرجات	البيانات المطلوبة	المرحلة
تحديد المسار القانوني للملف	اسم المحكمة ونوع القضية	مرحلة المحكمة
ربط الأطراف بالقضية	بيانات المدعي والمدعى عليه	مرحلة الأطراف
إكمال عناصر الإثبات عند الحاجة	تفاصيل الزواج، الشهود، الأبناء	مرحلة التفاصيل
تكوين محتوى الدعوى	شرح القضية والأسانيد القانونية	مرحلة الوقائع
JSON. إنشاء سجل القضية وإرجاع	Fetch/FormData إرسال النموذج عبر	مرحلة الحفظ

التقليدية وبعض التفاعل الديناميكي عبر Blade عند إضافة وتحديث وحذف القضية يدل على أن المشروع يدمج بين صفحات JSON وجود رد. وهذا يعطي تجربة أسرع للمستخدم عند الحفظ أو التعديل، JavaScript.

واجهات النظام وتجربة الاستخدام

تم تصميم واجهات المشروع باللغة العربية وباتجاه مناسب للمستخدم العربي. الصفحة الرئيسية تحمل هوية "نظام العدالة الذكي" وتوفر روابط واضحة. مدخل القضاة ولوحة الإدارة. كما أن لوحة التحكم تعرض منطقة بحث مباشرة وروابط سريعة لإضافة قضية أو عرض القضايا

أهم العناصر	الوصف	الواجهة
اسم النظام، هوية بصرية، روابط الدخول	الصفحة الرئيسية للنظام	welcome.blade.php
اسم المستخدم وكلمة المرور	دخول مستخدم النظام	auth/login.blade.php
بحث سريع وروابط عمليات	لوحة المستخدم	dashboard.blade.php
PDF نموذج متعدد المراحل ورفع	إضافة قضية	cases/create.blade.php
عرض القضايا مع البحث والإجراءات	قائمة القضايا	cases/index.blade.php
إحصاءات، رسوم بسيطة، تحليلات ذكية	لوحة القضاة	judge/dashboard.blade.php

نقطة قوة في تجربة الاستخدام

تقسيم نموذج إضافة القضية إلى مراحل يجعل النظام أسهل في العرض والمناقشة؛ لأنه يوضح أن المشروع يفهم طبيعة البيانات القضائية المتدرجة وليس مجرد نموذج إدخال بسيط

بالإضافة إلى ذلك، تحتوي الواجهات على تنسيق بصري مناسب مثل البطاقات، الأزرار، ألوان رسمية، وأيقونات، مما يعطي النظام طابعًا حكوميًا كامل للهواتف والأجهزة اللوحية responsive وقضائياً. ويمكن تطوير الواجهة لاحقًا بإضافة تصميم

المسؤول عن الحفظ، والنماذج المرتبطة بقاعدة البيانات Controller، تعد وحدة إدارة القضايا جوهر المشروع. فهي تربط بين صفحة إضافة القضية ثم يحفظ نوع القضية والوقائع والأسانيد القانونية وحالة القضية والمرفقات، TAIZ عند إنشاء قضية جديدة، يقوم النظام بتوليد رقم قضية يبدأ ببادئة

ملاحظات	التنفيذ داخل النظام	العملية
توليد رقم قضية وحفظ البيانات الأساسية	LegalCase::create داخل store	إضافة قضية
case_id كل طرف مرتبط بالقضية عبر	Party::create للمدعي والمدعى عليه	إضافة الأطراف
تضاف فقط عند وجود بيانات مرتبطة	MarriageDetail::updateOrCreate	تفاصيل الزواج
يدعم أكثر من شاهد للقضية	Witness::create داخل حلقة	إضافة الشهود
يدعم ملف فعلي وملخص نصي	ومحتوى مختصر PDF تخزين مسار	المرفقات
يسهل الوصول السريع للملفات	case_number أو اسم الطرف	البحث

عند بعض أنواع القضايا مثل إثبات الزواج أو النفقة. الفكرة هنا أن judiciary_records من النقاط المهمة في هذه الوحدة وجود مطابقة مع جدول النظام يحاول التحقق من وجود المدعى عليه في سجل قضائي سابق، ثم يضيف ملاحظة في رسالة النجاح. هذه الميزة تصلح كنواة لتكامل مستقبلي مع

سجلات، سمسة أو قواعد بيانات وطنية

تحسين مقترح

بدل تخزين اسم القاضي كنص؛ لأن الربط بالمفتاح judge_id أوسع لكل الحقول، وربط القضايا بالقضاة من خلال Validation ينصح بإضافة تحقق

الأجنبي يقلل الأخطاء ويحسن التقارير

توفر وحدة الإدارة إمكانية إدارة حسابات المستخدمين من خلال صفحات إنشاء وتعديل وحذف وعرض المستخدمين. تعتمد هذه الوحدة على عند حفظ كلمة المرور، وهذا جيد لأن كلمات المرور لا تحفظ كنص مباشر داخل قاعدة Hash وتستخدم AdminController وUserController البيانات عند إنشاء المستخدمين.

الوصف	الملف المسؤول	الوظيفة
إظهار قائمة الحسابات المسجلة	usersIndex / UserController@index	عرض المستخدمين
التحقق من الاسم وكلمة المرور ثم الحفظ	usersStore / UserController@store	إنشاء مستخدم
تحديث الاسم واسم المستخدم وكلمة المرور اختياريًا	usersUpdate / UserController@update	تعديل مستخدم
إزالة المستخدم من النظام	usersDestroy / UserController@destroy	حذف مستخدم
مدخل منفصل للإدارة	AdminController@login	دخول الإدارة

وجود لوحة إدارة منفصلة يعطي النظام قابلية للتحكم في الحسابات دون الرجوع إلى قاعدة البيانات مباشرة. وهذا مهم في الأنظمة المؤسسية لأن إدارة الحسابات يجب أن تكون من داخل النظام وبصلاحيات واضحة

ملاحظة أمنية مهمة

مع Clerk و Judge و Admin ينبغي عدم الاعتماد على كلمات مرور ثابتة داخل الكود لمدخل الإدارة أو القضاة. الأفضل بناء نظام أدوار وصلاحيات مثل Policies و Hash حفظ الحسابات في قاعدة البيانات واستخدام

لتسجيل من قام بإنشاء أو تعديل أو حذف المستخدمين والقضايا. هذه الإضافة مهمة جدًا في الأنظمة القضائية Audit Log كما يفضل إضافة سجل تدقيق لأنها تحفظ الأثر الإداري وتمنع التلاعب غير الموثق

لوحة القضاة والتحليل الذكي

تقدم لوحة القضاة جانبًا مميزًا في المشروع؛ إذ لا تكتفي بعرض القضايا، بل تحاول تحويل البيانات إلى مؤشرات تساعد على الفهم السريع. يقوم بحساب إجمالي القضايا، القضايا قيد النظر، القضايا المحكومة، توزيع القضايا حسب النوع، وتوزيعها على القضاة. ثم يتم تمرير JudgeController إلى `judge/dashboard.blade.php` هذه السانبات الواحدة

الفائدة	طريقة الحساب	المؤشر
قياس حجم العمل العام	<code>LegalCase::count()</code>	إجمالي القضايا
معرفة الضغط الحالي	قيد النظر <code>status =</code> فلترة	قضايا قيد النظر
قياس الإنجاز	محكومة <code>status =</code> فلترة	قضايا محكومة
تحديد أكثر أنواع القضايا انتشارًا	<code>groupBy(case_type)</code>	التصنيف حسب النوع
تقدير الأحمال على القضاة	<code>groupBy(judge_name)</code>	التوزيع على القضاة

وهي دالة تولد توصيات بناءً على قواعد مثل نسبة القضايا المتراكمة، أكثر نوع قضية، `generateAllInsights` أما جزء التحليل الذكي فيعتمد على. وليس نموذج تعلم آلي كامل، AI، تكرارًا، والتحذير عند زيادة القضايا الجديدة وقيد النظر. لذلك يمكن وصفه بأنه تحليل ذكي مبدئي أو محاكاة فرصة تطوير

يمكن مستقبلًا ربط هذه اللوحة بنموذج تنبؤ يحلل مدة الفصل المتوقعة، الأولوية، احتمالية التأخير، أو توصية بتوزيع القضايا بناءً على عبء كل قاضي ونوع التخصص.

الأمن والاختبار وملاحظات الجودة

الأمن عنصر أساسي في أي نظام قضائي، لأن البيانات المتداولة قد تكون حساسة وتشمل أسماء أطراف، أرقام هواتف، عناوين، مرفقات، ووقائع قانونية لكنه يحتاج إلى Hash للمستخدمين وحفظ كلمات المرور عبر Laravel Authentication المشروع يحتوي على أساس جيد من ناحية استخدام تحسينات قبل استخدامه في بيئة حقيقية

الإجراء المقترح	الوضع الحالي	المجال
roles/permissions. توحيد الدخول واعتماد	تسجيل دخول للمستخدمين وبعض مداخل منفصلة	المصادقة
نقل كل الحسابات لقاعدة البيانات	للمستخدمين، لكن توجد كلمات ثابتة لبعض Hash المداخل	كلمات المرور
كامل لقضايا الأطراف والمرفقات validation إضافة	موجود في إدارة المستخدمين جزئيًا	التحقق من المدخلات
التحقق من النوع والحجم وفحص الملفات	وحفظ المسار PDF رفع	المرفقات
للك عملية حساسة Audit Log إضافة	لا يظهر سجل تدقيق كامل	السجلات
النتيجة المتوقعة		اختبار مقترح
السماح بالدخول الصحيح ورفض البيانات الخاطئة		تسجيل دخول صحيح وخاطئ
إنشاء قضية مع أطراف وشهود ومرفق		إضافة قضية كاملة
ظهور القضية المرتبطة بالطرف		البحث باسم طرف
تحديث الحالة وظهورها في لوحة القضية		تعديل حالة القضية
حذف القضية والبيانات التابعة لها		حذف قضية

متعلق بتحميل بعض الامتدادات في بيئة تشغيل محمولة. هذا لا يعني بالضرورة فشل المشروع، لكنه يدل على أهمية PHP كما لوحظ وجود سجل أخطاء وغيرها mbstring و openssl و sqlsrv اللازمة مثل PHP توثيق بيئة التشغيل المطلوبة والتأكد من توفر امتدادات

فكرة جيدة ومناسبة لرقمنة العمل القضائي، خصوصًا في جانب إدارة القضايا وربطها بالأطراف والمرفقات والمؤشرات. وقد أظهر JSJS يمثل مشروع ومسارات تشغيل Views وMigrations وModels وControllers فحص الملفات أن المشروع لا يقتصر على واجهة شكلية، بل يحتوي على فعالية. هذا يجعله مشروعًا قابلاً للعرض والمناقشة، وقابلًا للتطوير إلى نظام أكثر نضجًا

تنظيم قاعدة البيانات، وجود لوحة قضاء تحليلية، وجود إدارة مستخدمين، ودعم Laravel أهم نقاط القوة في المشروع هي وضوح الفكرة، استخدام إطار. كما أن إضافة مشغل محلي للويندوز تعطي ميزة عملية عند تشغيل النظام في بيئة عرض أو جهاز مستقل. PDF مرفقات

الأولوية	التوصية
عالية	استبدال كلمات المرور الثابتة بنظام صلاحيات وأدوار مبني على قاعدة البيانات
عالية	إضافة تحقق شامل للمدخلات وحماية أفضل للمرفقات
متوسطة	وفهارس بحث لتحسين الأداء مع زيادة عدد القضايا pagination إضافة
متوسطة	لتصبح مبنية على بيانات تاريخية ونماذج تنبؤية فعلية AI تطوير لوحة
منخفضة	تحسين التصميم المتجاوب وإضافة توثيق مستخدم مختصر

خلاصة نهائية

النظام مناسب كنموذج مشروع تخرج أو مشروع جامعي عملي، ويعرض فكرة واضحة عن التحول الرقمي في قطاع العدالة. ومع معالجة الجوانب الأمنية والتشغيلية، يمكن أن يصبح نواة لنظام قضائي أكثر تكاملاً

Composer ملفات، Blade Views، Migrations، Models، Controllers، Laravel routes. مراجع التقرير: ملفات المشروع المرفقة، وتشمل وملفات التشغيل المحلية، Package و